

WEEK-04

Stone Game-One Four

Problem Statement:

Alice and Bob are playing a game called "Stone Game". Stone game is a two-player game. Let N be the total number of stones. In each turn, a player can remove either one stone or four stones. The player who picks the last stone, wins. They follow the "Ladies First" norm. Hence Alice is always the one to make the first move. Your task is to find out whether Alice can win, if both play the game optimally.

Input Format

First line starts with T , which is the number of test cases. Each test case will contain N number of stones.

Output Format

Print "Yes" in the case Alice wins, else print "No".

Constraints $1 \leq T \leq 1000$ $1 \leq N \leq 10000$

Sample Input

3

1

6

7

Sample Output

Yes

Yes

No

```

1  #include<stdio.h>
2  int main()
3  {
4      int T,N,rem,mod;
5      scanf("%d", &T);
6      while (T--)
7      {
8          scanf("%d",&N);
9          if(N%4 == 0)
10         {
11             rem = N/4;
12             if(rem% 2 != 0)
13             {
14                 printf("Yes\n");
15             }
16             else
17             {
18                 printf("No\n");
19             }
20         }
21         else
22         {
23             rem = N/4;
24             mod = N%4;
25             if(rem% 2!= 0 )
26             {
27                 if (mod == 1 || mod == 3)
28                 {
29                     printf("No\n");
30                 }
31                 else
32                 {
33                     printf("Yes\n");
34                 }
35             }
36             else
37             {
38                 if(mod == 1 || mod == 3)
39                 printf("Yes\n");
40                 else
41                 printf("No\n");
42             }
43         }
44     }
45     return 0;
46 }

```

	Input	Expected	Got	
✓	3	Yes	Yes	✓
	1	Yes	Yes	
	6	No	No	
	7			

Passed all tests! ✓

Holes in a Number

Problem Statement:

You are designing a poster which prints out numbers with a unique style applied to each

of them. The styling is based on the number of closed paths or holes present in a given number.

The number of holes that each of the digits from 0 to 9 have are equal to the number of closed paths in the digit. Their values are:

1, 2, 3, 5, 7 = 0 holes.

0, 4, 6, 9 = 1 hole.

8 = 2 holes.

Given a number, you must determine the sum of the number of holes for all of its digits.

For example, the number 819 has 3 holes.

Complete the program, it must return an integer denoting the total number of holes in num.

Constraints

$1 \leq \text{num} \leq 109$

Input Format For Custom Testing

There is one line of text containing a single integer num, the value to process.

Sample Input

630

Sample Output

2

```
1 #include<stdio.h>
2 int countHoles(int num)
3 {
4     int holes = 0;
5     while (num > 0)
6     {
7         int digit = num % 10;
8         switch (digit)
9         {
10             case 0:
11             case 4:
12             case 6:
13             case 9:
14                 holes += 1;
15                 break;
16             case 8:
17                 holes += 2;
18                 break;
19         }
20         num /= 10;
21     }
22     return holes;
23 }
24 int main()
25 {
26     int num;
27     scanf("%d", &num);
28     int result = countHoles(num);
29     printf("%d\n", result);
30     return 0;
31 }
```

	Input	Expected	Got	
✓	630	2	2	✓
✓	1288	4	4	✓

Passed all tests! ✓

Philaland Coin

Problem Statement:

The problem solvers have found a new Island for coding and named it as Philaland. These smart people were given a task to make a purchase of items at the Island easier by distributing various coins with different values. Manish has come up with a solution that if we make coins category starting from \$1 till the maximum price of the item present on Island, then we can purchase any item easily. He added the following example to prove his point.

Let's suppose the maximum price of an item is 5\$ then we can make coins of {\$1, \$2, \$3, \$4, \$5} to purchase any item ranging from \$1 till \$5.

Now Manisha, being a keen observer suggested that we could actually minimize the number of coins required and gave following distribution {\$1, \$2, \$3}. According to him any item can be purchased one time ranging from \$1 to \$5. Everyone was impressed with both of them. Your task is to help Manisha come up with a minimum number of denominations for any arbitrary max price in Philaland.

Input Format

Contains an integer N denoting the maximum price of the item present on Philaland.

Output Format

Print a single line denoting the minimum number of denominations of coins required.

Constraints

$1 \leq T \leq 100$ $1 \leq N \leq 5000$

Sample Input 1:

10

Sample Output 1:

4

```

1 #include<stdio.h>
2 int minDenominations(int N)
3 {
4     int count = 0;
5     int sum = 0;
6     int denomination = 1;
7     while (sum < N)
8     {
9         sum += denomination;
10        denomination *= 2;
11        count++;
12    }
13    return count;
14 }
15 int main()
16 {
17     int N;
18     scanf("%d", &N);
19     int result = minDenominations(N);
20     printf("%d\n", result);
21     return 0;
22 }

```

	Input	Expected	Got	
✓	10	4	4	✓
✓	5	3	3	✓
✓	20	5	5	✓
✓	500	9	9	✓
✓	1000	10	10	✓

Passed all tests! ✓

Number Count

Problem Statement:

A set of N numbers (separated by one space) is passed as input to the program. The program must identify the count of numbers where the number is odd number.

Input Format:

The first line will contain the N numbers separated by one space.

Boundary Conditions:

$3 \leq N \leq 50$

The value of the numbers can be from -999999999 to 999999999

Output Format:

The count of numbers where the numbers are odd numbers.

Sample Input:

241001122

5 10 15 20 25 30 35 40 45 50

Sample Output:

5

```
1 #include<stdio.h>
2 int main()
3 {
4     int N=10,i=0,a,sum=0;
5     while(i<N)
6     {
7         scanf("%d", &a);
8         if(a%2 != 0)
9         {
10             sum += 1;
11         }
12         i++;
13     }
14     printf("%d",sum);
15     return 0;
16 }
```

	Input	Expected	Got	
✓	5 10 15 20 25 30 35 40 45 50	5	5	✓

Passed all tests! ✓

Confusing Number

Problem Statement:

Given a number N, return true if and only if it is a *confusing number*, which satisfies the following condition:
We can rotate digits by 180 degrees to form new digits. When 0, 1, 6, 8, 9 are rotated 180 degrees, they become 0, 1, 9, 8, 6 respectively. When 2, 3, 4, 5 and 7 are rotated 180 degrees, they become invalid. A *confusing number* is a number that when rotated 180 degrees becomes a **different** number with each digit valid.

Example 1:

6 → 9

Input: 6
Output: true
Explanation: We get 9 after rotating 6, 9 is a valid number and 9 != 6.

Example 2:

89 → 68

Input: 89
Output: true
Explanation: We get 68 after rotating 89, 86 is a valid number and 86 != 89.

Example 3:

11 → 11

Input: 11
Output: false
Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number.

Example 4:

25 → 25

Input: 25
Output: false
Explanation: We get an invalid number after rotating 25.

Note:

- 0 <= N <= 10^9
- After the rotation we can ignore leading zeros, for example if after rotation we have 0008 then this number is considered as just 8.

```

1 #include<stdio.h>
2 #include<stdbool.h>
3 bool isConfusingNumber(int N)
4 {
5     int rotatedDigits[]={0,1,-1,-1,-1,-1,9,-1,8,6};
6     int original = N;
7     int rotatedNumber = 0;
8     int placeValue = 1;
9     while(N>0)
10    {
11        int digit = N%10;
12        if(rotatedDigits[digit]==-1)
13        {
14            return false;
15        }
16        rotatedNumber += rotatedDigits[digit]*placeValue;
17        placeValue *= 10;
18        N/=10;
19    }
20    return rotatedNumber != original;
21 }
22 int main()
23 {
24     int N;
25     scanf("%d", &N);
26     if(isConfusingNumber(N))
27     {
28         printf("true\n");
29     }
30     else
31     {
32         printf("false\n");
33     }
34     return 0;
35 }

```

	Input	Expected	Got	
✓	6	true	true	✓
✓	89	true	true	✓
✓	25	false	false	✓

Passed all tests! ✓

Nutrition Value

Problem Statement:

A nutritionist is labeling all the best power foods in the market. Every food item arranged in a single line, will have a value beginning from 1 and increasing by 1 for each, until all items have a value associated with them. An item's value is the same as the number of macronutrients it has. For example, food item with value 1 has 1 macronutrient, food item with value 2 has 2 macronutrients, and incrementing in this fashion.

The nutritionist has to recommend the best combination to patients, i.e. maximum

total of macronutrients. However, the nutritionist must avoid prescribing a particular sum of macronutrients (an 'unhealthy' number), and this sum is known. The nutritionist chooses food items in the increasing order of their value. Compute the highest total of macronutrients that can be prescribed to a patient, without the sum matching the given 'unhealthy' number. Here's an illustration: Given 4 food items (hence value: 1,2,3 and 4), and the unhealthy sum being 6 macronutrients, on choosing items 1, 2, 3 -> the sum is 6, which matches the 'unhealthy' sum. Hence, one of the three needs to be skipped. Thus, the best combination is from among: • $2 + 3 + 4 = 9$ • $1 + 3 + 4 = 8$ • $1 + 2 + 4 = 7$ Since $2 + 3 + 4 = 9$, allows for maximum number of macronutrients, 9 is the right answer. Complete the code in the editor below. It must return an integer that represents the maximum total of macronutrients, modulo 1000000007 ($10^9 + 7$). It has the following:

n: an integer that denotes the number of food items
k: an integer that denotes the unhealthy number

Constraints

$$1 \leq n \leq 2 \times 10^9 \quad 1 \leq k \leq 4 \times 10^{15}$$

Input Format For Custom Testing

The first line contains an integer, n, that denotes the number of food items. The second line contains an integer, k, that denotes the unhealthy number.

Sample Input 0

2

2

Output 0

3

```

1 #include<stdio.h>
2 int main()
3 {
4     long n,k,sum=0;
5     scanf("%ld %ld", &n, &k);
6     for (int i=1; i<=n; i++)
7     {
8         sum+=i;
9         if (sum == k)
10        {
11            sum = sum -1;
12        }
13    }
14    printf("%ld",sum%1000000007);
15    return 0;
16 }
```

	Input	Expected	Got	
✓	2 2	3	3	✓
✓	2 1	2	2	✓
✓	3 3	5	5	✓